

While grouping, we can also negate the match condition for a group by suffixing the text match condition with the `!` operator, like this: `/TEXT/ !{ statements }`. Such a group is executed whenever the pattern does *not* match `/TEXT/`.

Read and write from files using Sed

We can read or write to an intermediate file by specifying file-names and match/replace text in the `sed` script. Let us look at an example. We set up files as follows (file name followed by content):

```
file1.txt:
A line from file1
file2.txt:
A line from file2
list.txt:
FILE1
```

The file `list.txt` is our input file. While reading from it, if a particular match occurs, and if we wish to read in the contents of another file to replace the matched pattern, we can issue the following code:

```
$ cat list.txt | sed '/FILE1/ { r file1.txt
d }
/FILE2/ { r file2.txt
d }'
```

`r filename` is the command used. Note that when `read (r)` is used after the filename, there should be a new line. That is why, instead of continuing the command on the same line, later commands are given on new lines. Here, `d` is used to remove the matched text read from `list.txt`. Experiment with this: remove the 'd' commands; edit `list.txt` and insert 'FILE2' on the first line (before the 'FILE1' line)... run the command after each of your changes and see what effect it has.

We can write conditionally matched patterns or replaced text to a file, as follows:

```
$ seq 100 | sed -n '30,33 { w extracted.txt
}'
$ cat extracted.txt
30
```

31
32
33

Like the `read` command '`r`' operates on files, we can use the write command '`w`' to write matched text to a specified file. In the above `sed` script, it matches line numbers 30 to 33, which are written into a file named `extracted.txt`. As before, the file-name is immediately followed by a newline. The `-n` option is given to prevent `Sed` from printing input lines to standard output.

Quit operator

`Sed` reads input line by line and processes it. We can stop the execution of `Sed` at some particular matching line and quit, so that the following lines will not be read and processed, using the '`q`' command. For example, we quit this script after line 3:

```
$ seq 10 | sed '3 q'
1
2
3
```

In this article, we have come across basic concepts and building blocks that can be used to solve complex problems. By making proper use of what we have learned in this article, it is possible to break up text-processing tasks into smaller, simpler tasks, and solve them. `Sed` has more interesting features and data structures that add more text-processing capabilities. We will learn about data structures, operators and a few text-processing problems in the second part of this article, next month. Happy hacking, till we meet again! 

By: Sarath Lakshman

The author is a hacktivist of Free and Open Source Software from Kerala. He loves working on the GNU/Linux environment and contributes to the Pardus Linux distro project. He has recently authored the Linux Shell Scripting Cookbook, which gives insights into shell scripting using 119 'recipes'. He can be reached via his website <http://www.sarathlakshman.com>.

TechnoMail - Enterprise Email Server
Anti SPAM, Anti Virus, Email Content Filtering

Firewall, Internet Access Control
Content Filtering, Site Blocking

Bandwidth Management System

Managed Email Hosting Solutions

TechnoInfotech®

1, Vikas Permisses, 11 Bank Street, Fort, Mumbai, India - 400 001. Mobile: 09167399917. info@technoinfotech.com